# VULNERABILITY ASSESSMENT REPORT

## Web Application Security Audit

**Target**

**http://demo.testfire.net (Altoro Mutual)**

Scope: Read-Only Passive Assessment

Date: April 13, 2026

**Classification: Confidential**

Prepared By: Khatri Mann Jatinkumar

Organization: Future Interns

Email: **mannkhatri2464@gmail.com** | **LinkedIn**

# 1. Executive Summary

This Vulnerability Assessment Report presents the findings from a passive, read-only security audit conducted on http://demo.testfire.net, a demonstration banking web application operated by IBM (Altoro Mutual). The assessment was performed strictly within ethical guidelines - no exploitation, brute force, or denial-of-service activities were performed.

The assessment identified 12 vulnerabilities across critical, high, and medium severity levels. These findings pose significant risks to the confidentiality, integrity, and availability of the application and its users' data.

| Critical | High | Medium | Total Findings |
|---|---|---|---|
| 3 | 6 | 3 | 12 |

# 2. Scope & Methodology

## 2.1 Target Information

| | |
|---|---|
| **Website** | http://demo.testfire.net |
| **Application** | Altoro Mutual (IBM Demo Banking App) |
| **IP Address** | 65.61.137.117 |
| **Server** | Apache-Coyote/1.1 |
| **Assessment Date** | April 13, 2026 |
| **Assessment Type** | Passive / Read-Only Vulnerability Assessment |

## 2.2 Scope

- Public-facing pages only
- Passive scanning and header analysis
- Browser-based inspection (DevTools, Cookies)
- No login bypass, exploitation, or brute force

## 2.3 Tools Used

- SecurityHeaders.com - HTTP security header analysis
- Browser DevTools (Chrome) - Network & cookie inspection
- Manual testing - XSS and SQL Injection detection

## 2.4 Ethical Guidelines

This assessment strictly followed ethical guidelines as defined in the task scope. Only passive, read-only techniques were used. No data was modified, extracted, or damaged. The target site is an intentionally vulnerable demo application authorized for security testing.

## 3. Vulnerability Findings

The following table summarizes all identified vulnerabilities, their risk classification, and the location where they were discovered.

| Sr. | Vulnerability | Risk | Location | Category |
|---|---|---|---|---|
| 1 | **Missing Content-Security-Policy** | **High** | Security Headers | Misconfiguration |
| 2 | **Missing X-Frame-Options** | **High** | Security Headers | Misconfiguration |
| 3 | **Missing X-Content-Type-Options** | **Medium** | Security Headers | Misconfiguration |
| 4 | **Missing Referrer-Policy** | **Medium** | Security Headers | Misconfiguration |
| 5 | **Missing Permissions-Policy** | **Medium** | Security Headers | Misconfiguration |
| 6 | **Server Version Disclosed** | **High** | HTTP Response | Information Exposure |
| 7 | **No HTTPS Enforcement** | **High** | Transport Layer | Insecure Transmission |
| 8 | **Missing SameSite Cookie Attribute** | **High** | Session Cookie | Session Management |
| 9 | **Session Cookie Never Expires** | **High** | Session Cookie | Session Management |
| 10 | **Cross-Site Scripting (XSS)** | **Critical** | Search Box | Injection |
| 11 | **SQL Injection - Auth Bypass** | **Critical** | Login Page | Injection |
| 12 | **Broken Access Control** | **Critical** | Bank Pages | Access Control |

# 4. Detailed Findings

## Finding 10 - Cross-Site Scripting (XSS) | CRITICAL

| | |
|---|---|
| **Vulnerability** | Cross-Site Scripting (XSS) |
| **Risk Level** | Critical |
| **Location** | Search box - http://demo.testfire.net |
| **OWASP Category** | A03:2021 - Injection |

**Description:**

The website's search functionality does not sanitize user input. A malicious JavaScript payload injected into the search field was executed by the browser, confirming a reflected XSS vulnerability.

**Proof of Concept:**

```
Payload used: <script>alert('This site is vulnerable!')</script>
```

**Business Impact:**

- Attackers can steal session cookies and hijack user accounts
- Malicious scripts can redirect users to phishing sites
- Sensitive banking data can be captured and exfiltrated

**Remediation:**

- Implement input validation and output encoding on all user inputs
- Deploy a Content Security Policy (CSP) header
- Use a Web Application Firewall (WAF)

## Finding 11 - SQL Injection (Authentication Bypass) | CRITICAL

| | |
|---|---|
| **Vulnerability** | SQL Injection - Authentication Bypass |
| **Risk Level** | Critical |
| **Location** | Login page - /bank/login.aspx |
| **OWASP Category** | A03:2021 - Injection |

**Description:**

The login form is vulnerable to SQL injection. By entering a crafted SQL payload as the username, an attacker can bypass authentication entirely without knowing a valid password.

**Proof of Concept:**

```
Username: admin' or '10' = '10    Password: 123456789
```

**Business Impact:**

- Complete authentication bypass - anyone can log in as admin
- Full access to customer banking accounts and data
- Potential for database extraction, modification, or deletion

**Remediation:**

- Use parameterized queries / prepared statements
- Never concatenate user input directly into SQL queries
- Implement input validation and least privilege database accounts

## Finding 12 - Broken Access Control | CRITICAL

| | |
|---|---|
| **Vulnerability** | Broken Access Control |
| **Risk Level** | Critical |
| **Location** | /bank/ directory - multiple pages |
| **OWASP Category** | A01:2021 - Broken Access Control |

**Description:**

Sensitive banking pages under the /bank/ directory are accessible without authentication. This allows unauthenticated users to reach pages that should require login.

**Business Impact:**

- Unauthorized access to sensitive banking functionality
- Violation of data privacy and compliance regulations
- Could expose customer financial information

**Remediation:**

- Implement server-side authentication checks on all protected pages
- Redirect unauthenticated users to the login page
- Apply principle of least privilege across all routes

## Finding 6 - Server Version Disclosed | HIGH

| | |
|---|---|
| **Vulnerability** | Server Version Information Disclosure |
| **Risk Level** | High |
| **Location** | HTTP Response Header -Server field |
| **Evidence** | Server: Apache-Coyote/1.1 |

**Description:**

The server reveals its exact software and version (Apache-Coyote/1.1) in the HTTP response headers. This allows attackers to search for known vulnerabilities specific to that version.

**Remediation:**

- Configure the server to suppress or obfuscate the Server header
- Upgrade server software to the latest stable version

## 5. Security Header Analysis

Security headers are HTTP response headers that instruct the browser on how to behave when handling the website's content. The following headers are missing from http://demo.testfire.net:

| Missing Header | Risk | Impact |
|---|---|---|
| **Content-Security-Policy** | **High** | Allows XSS and code injection attacks |
| **X-Frame-Options** | **High** | Site can be embedded in iframes - Clickjacking risk |
| **X-Content-Type-Options** | **Medium** | Browser may misinterpret file types |
| **Referrer-Policy** | **Medium** | Sensitive URLs may leak to third-party sites |
| **Permissions-Policy** | **Medium** | Camera, microphone access not restricted |

# 6. Remediation Recommendations

The following remediation steps are prioritized by risk level. Critical and High issues should be addressed immediately.

## Immediate Actions (Critical)

- Fix SQL Injection: Replace all dynamic SQL queries with parameterized queries or prepared statements
- Fix XSS: Implement input validation and output encoding on all user-controlled inputs
- Fix Broken Access Control: Add server-side authentication checks to all protected routes

## Short-Term Actions (High)

- Enable HTTPS: Install an SSL/TLS certificate and enforce HTTPS across the entire site
- Add Security Headers: Implement Content-Security-Policy and X-Frame-Options headers
- Fix Session Management: Add SameSite=Strict attribute to all session cookies and implement session timeout
- Hide Server Information: Configure the web server to suppress version details in response headers

## Medium-Term Actions (Medium)

- Add X-Content-Type-Options: nosniff header to all responses
- Add Referrer-Policy: strict-origin-when-cross-origin header
- Add Permissions-Policy header to restrict access to browser features

## 7. Conclusion

The security assessment of http://demo.testfire.net revealed a total of 12 vulnerabilities including 3 Critical, 6 High, and 3 Medium severity issues. The most severe findings - SQL Injection, Cross-Site Scripting, and Broken Access Control - represent fundamental security weaknesses that could lead to complete compromise of the application and its users' data in a real-world scenario.

The absence of basic security headers, lack of HTTPS enforcement, and insecure session management further compound the overall risk posture of the application.

It is strongly recommended that the development and security teams address all Critical and High severity findings immediately, followed by Medium severity issues in the next development cycle. A follow-up assessment should be conducted after remediation to verify that vulnerabilities have been properly resolved.

End of Report

Prepared by: Cyber Security Intern | Future Interns | April 2026